

Design Patterns

C# Developer Reference — Session 2

Singleton · Factory · Observer

Quick Reference

Pattern	Category	Problem it solves	Key idea
Singleton	Creational	Need exactly one instance app-wide	Private constructor + static instance
Factory	Creational	Object creation scattered everywhere	One place creates all objects
Observer	Behavioral	Notify many objects when state changes	Subscribe / Unsubscribe / Notify

■ *You don't need to design patterns from scratch. Write working code first. When you notice code is hard to change or test — that's when patterns help. Patterns are weapons to reach for when you recognise the problem, not rules to apply upfront.*

1. Singleton Pattern

"Ensure a class has only one instance throughout the entire application."

Some things should only exist once — a database connection, a logger, app config. Creating multiple instances wastes resources and can cause data conflicts.

Three rules of Singleton:

- Private constructor — nobody outside can call `new Logger()`
- Static instance — lives at class level, not object level
- Public static property — the only way to get the instance

■ Without Singleton

```
// ■ BAD – new instance created every time, no control

var logger1 = new Logger();

var logger2 = new Logger();

// logger1 and logger2 are different objects – wasteful and inconsistent
```

■ Singleton with Lazy — thread-safe, created only once

```
public class DatabaseConnection

{

    // Lazy<T> – does not create the object until first use

    // readonly – cannot be reassigned after declaration

    // static – belongs to the class, not to any instance

    private static readonly Lazy<DatabaseConnection> _instance =

        new Lazy<DatabaseConnection>(() => new DatabaseConnection());

    // Private constructor – blocks anyone calling new DatabaseConnection()

    // Empty body is fine – its only job is to be private

    private DatabaseConnection()
```

[illegible]

■ *Lazy is a wrapper. It holds your object inside .Value. Think of it as a box — empty at first. The moment you open it (.Value) for the first time, the object is created and stored. Every open after that returns the same object already inside.*

2. Factory Pattern

"Centralise object creation — one place creates all objects of a family."

When new Dog(), new Cat(), new Bird() are scattered across 50 files, adding a new animal means hunting through all 50. Factory brings all creation logic to one place.

■ Without Factory — creation scattered everywhere

■ Factory — one place for all creation logic

```
// Shared contract for all animals

public interface IAnimal { void MakeSound(); }

public class Dog : IAnimal { public void MakeSound() => Console.WriteLine("Woof!"); }

public class Cat : IAnimal { public void MakeSound() => Console.WriteLine("Meow!"); }

public class Bird : IAnimal { public void MakeSound() => Console.WriteLine("Tweet!"); }

// ■■■ Basic Factory ■■■
```

[illegible]

3. Observer Pattern

"When one object changes, all objects watching it are notified automatically."

Like subscribing to a YouTube channel. When a new video is uploaded, every subscriber gets a notification. The channel does not know or care who the subscribers are — it just calls `Notify()` and each subscriber handles it.

Two roles:

- Subject (YoutubeChannel) — the object being watched. Manages the subscriber list.
- Observer (EmailSubscriber, SmsSubscriber) — reacts when the subject changes.

■ Without Observer — tightly coupled to every notifier

```
// ■ BAD – channel hardwired to specific notifiers

public class YoutubeChannel

{

    public void UploadVideo(string title)

    {

        new EmailNotifier().Notify(title);    // tightly coupled

        new SmsNotifier().Notify(title);      // adding AppNotifier? edit here

    }

}
```

■ Observer — loosely coupled, add subscribers without touching the channel

```
// Contract for all observers – anyone who wants notifications implements this

public interface IObserver { void Update(string message); }

// Contract for the subject – defines subscribe / unsubscribe / notify

public interface ISubject

{
```



```

    void Subscribe(IObserver observer);

    void Unsubscribe(IObserver observer);

    void Notify(string message);
}

// The subject – does not know who its observers are, only that they exist

public class YoutubeChannel : ISubject
{
    private readonly List<IObserver> _observers = new();

    public void Subscribe(IObserver o) => _observers.Add(o);

    public void Unsubscribe(IObserver o) => _observers.Remove(o);

    public void Notify(string message)
    {
        foreach (var o in _observers)
        {
            o.Update(message);    // does not care what type each observer is
        }
    }

    public void UploadVideo(string title)
    {
        Console.WriteLine($"Uploaded: {title}");

        Notify($"New video: {title}");    // automatically fires all observers
    }
}

```

[illegible]

```
channel.Subscribe(user1);

channel.Subscribe(user2);

channel.Subscribe(user3);


channel.UploadVideo("Learning C# Session 1");

// Uploaded: Learning C# Session 1

// Email -> trung@gmail.com: New video: Learning C# Session 1

// SMS    -> 0901234567: New video: Learning C# Session 1

// Email -> nam@gmail.com: New video: Learning C# Session 1


channel.Unsubscribe(user2); // user2 opts out


channel.UploadVideo("Learning C# Session 2");

// Email -> trung@gmail.com: New video: Learning C# Session 2

// Email -> nam@gmail.com: New video: Learning C# Session 2

// user2 receives nothing – already unsubscribed
```

■ *Observer connects directly to SOLID: D — YoutubeChannel depends on IObserver (abstraction), not EmailSubscriber or SmsSubscriber (concrete). O — Add a new subscriber type (AppNotifier) by creating a new class, zero changes to YoutubeChannel.*

Patterns vs SOLID

Design patterns are not separate from SOLID — they are practical implementations of SOLID principles.

Pattern	SOLID connection	Real-world use case
Singleton	S — one responsibility: manage the single instance	DB connection, Logger, Config
Factory	O — extend by adding classes, not editing Create() D — caller depends on IAnimal, not Dog/Cat directly	Creating objects of a family
Observer	O — add subscribers without touching the subject D — subject depends on IObserver, not concrete types	Event systems, notifications

Write working code first. Recognise the problem. Then reach for the pattern.